

CS 240 - Lab Midsem

TAs: Aditya, Yash, Niral & Malay

Spring 2025

Instructions:

- This lab has 3 questions. All the questions are graded. The maximum score is 100.
- Create a folder on your Desktop with the following directory structure. The folder must contain only the specified files:

```
submission_<roll_number>/  
|-- q1/  
    |--classifiers.py  
|-- q2/  
    |--tree.py  
|-- q3/  
    |--model.py
```

- Once you have completed the work, open the terminal, type the following command, and then call a TA: `check-cs240`
- The TA will run `submit-cs240` to submit your work to the server. **It is your responsibility to ensure your work is submitted.** If you fail to inform us to submit your work, you will not be graded.

Q1. Mini-batch Gradient Descent with ℓ_4 Regularization [25 marks]

After being released from the prison realm, Satoru Gojo forgot his favourite type of regularization. After seeing that ℓ_1 and ℓ_2 regularization being used by Sukuna and Geto, he seems intent on using ℓ_4 regularization.

You are given a binary classification dataset

$$\{(x_i, y_i)\}_{i=1}^n, \quad y_i \in \{+1, -1\}.$$

We consider a linear classifier

$$f(x) = w^\top x + b.$$

Objective Function

Train the classifier by minimizing the regularized logistic loss

$$L(w, b) = \sum_{i=1}^n \log(1 + e^{-y_i f(x_i)}) + \frac{\lambda}{4} \|w\|_4^4,$$

where

$$\|w\|_4^4 = \sum_{j=1}^d w_j^4,$$

and $\lambda > 0$ is the regularization parameter.

Mini-batch Objective

For a mini-batch $\mathcal{B}$,

$$L(w, \mathcal{B}) = \sum_{i \in \mathcal{B}} \log(1 + e^{-y_i f(x_i)}) + \frac{\lambda}{4} \sum_{j=1}^d w_j^4.$$

Tasks

1. Clean up the dataset by first standardizing each feature j .
2. Derive the gradients of $L_{\mathcal{B}}(w, b)$ with respect to w and b .
3. To perform mini-batch gradient descent, shuffle the data and then iterate over B -sized folds of the data. (In each epoch you call step on each B sized fold of the data)
4. Train the classifier for different values of λ and observe training loss and classification accuracy on the validation set.
5. Fill in this value in the `give_optimal_hyperparameter` function by sweeping over λ on the validation set to compute optimal λ .

Submission

Submit your modified `model.py` file.

Majority of the marks of this question will be awarded based on overall correctness and not function-level correctness.

Q2. Regression Trees: Local Linear Estimators [40 marks]

In previous assignments, you explored the standard Classification and Regression Tree (CART) framework, where the prediction at any leaf node is the arithmetic mean of the target values within that partition. This results in an inefficient step-function approximation for continuous trends.

In this assignment, we implement a variation of the Regression Tree. Instead of predicting a constant mean, every node fits a *local linear regressor* using the closed-form least-squares solution. By centering the data, the bias term is constrained to be the mean of the target values, and the regression weights capture the localized continuous slopes.

Theory and Node Representation

A Regression Tree processes a dataset recursively. Let

$$D_i = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^{N_i}$$

be the subset of N_i training examples routed to node i , where $\mathbf{x}^{(n)} \in \mathbb{R}^d$ is a column feature vector and $y^{(n)} \in \mathbb{R}$ is the corresponding scalar target.

Each node is formally represented by the tuple:

$$\mathcal{N}_i = (D_i, \mathbf{w}_i, \bar{\mathbf{x}}_i, \bar{y}_i) \quad (1)$$

where:

- D_i is the partitioned dataset at node i .
- $\bar{\mathbf{x}}_i = \frac{1}{N_i} \sum_{n=1}^{N_i} \mathbf{x}^{(n)} \in \mathbb{R}^d$ is the empirical mean vector of the features.
- $\bar{y}_i = \frac{1}{N_i} \sum_{n=1}^{N_i} y^{(n)} \in \mathbb{R}$ is the empirical mean of the targets.
- $\mathbf{w}_i \in \mathbb{R}^d$ is the optimal weight vector for the local linear model.

The local prediction for a query vector $\mathbf{x}$ routed to node i operates on mean-centered features:

$$\hat{y} = (\mathbf{x} - \bar{\mathbf{x}}_i)^T \mathbf{w}_i + \bar{y}_i \quad (2)$$

Node Impurity and Regularized Loss

To evaluate the quality of a partition, the impurity of a node is defined as the Least-Squares Loss augmented with an L2 regularization (Ridge) penalty, controlled by a strictly positive scalar $\lambda > 0$.

Summation Formulation. The loss $L(\mathcal{N}_i, \mathbf{w})$ evaluated over D_i is:

$$L(\mathcal{N}_i, \mathbf{w}) = \sum_{n=1}^{N_i} \left(y^{(n)} - \bar{y}_i - (\mathbf{x}^{(n)} - \bar{\mathbf{x}}_i)^T \mathbf{w} \right)^2 + \lambda \|\mathbf{w}\|_2^2 \quad (3)$$

Vectorized Formulation. Define the following mean-centered matrices and vectors:

- $\mathbf{X}_i \in \mathbb{R}^{N_i \times d}$: the mean-centered design matrix whose n -th row is $(\mathbf{x}^{(n)} - \bar{\mathbf{x}}_i)^T$.

- $\mathbf{y}_i \in \mathbb{R}^{N_i}$: the mean-centered target vector whose n -th entry is $y^{(n)} - \bar{y}_i$.

The vectorized loss is then:

$$L(\mathcal{N}_i, \mathbf{w}) = \|\mathbf{y}_i - \mathbf{X}_i \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_2^2 \quad (4)$$

We further define the following sufficient statistics, all computed from the centered quantities:

$$S_{xx} = \mathbf{X}_i^T \mathbf{X}_i \in \mathbb{R}^{d \times d}, \quad S_{xy} = \mathbf{X}_i^T \mathbf{y}_i \in \mathbb{R}^d, \quad S_{yy} = \mathbf{y}_i^T \mathbf{y}_i = \sum_{n=1}^{N_i} (y^{(n)} - \bar{y}_i)^2 \in \mathbb{R}.$$

The closed-form minimizer $\mathbf{w}_i$ is obtained by solving the regularized normal equations:

$$\mathbf{w}_i = (S_{xx} + \lambda I)^{-1} S_{xy} = (\mathbf{X}_i^T \mathbf{X}_i + \lambda I)^{-1} \mathbf{X}_i^T \mathbf{y}_i \quad (5)$$

Substituting $\mathbf{w}_i$ back into the objective simplifies the minimized node impurity to:

$$L^*(\mathcal{N}_i) = S_{yy} - \mathbf{w}_i^T S_{xy} = \mathbf{y}_i^T \mathbf{y}_i - \mathbf{w}_i^T \mathbf{X}_i^T \mathbf{y}_i \quad (6)$$

A derivation of this simplification is provided in Appendix A.

Recursive Splitting and Optimization

The CART algorithm extends a node by partitioning D_i into two mutually exclusive subsets, D_L and D_R , based on a feature index $j \in \{1, \dots, d\}$ and a numerical threshold $t \in \mathbb{R}$.

Set Formulation.

$$D_L(j, t) = \{(\mathbf{x}^{(n)}, y^{(n)}) \in D_i \mid x_j^{(n)} \leq t\}$$

$$D_R(j, t) = \{(\mathbf{x}^{(n)}, y^{(n)}) \in D_i \mid x_j^{(n)} > t\}$$

where $x_j^{(n)}$ denotes the j -th feature of the n -th example.

Vectorized Masking. Let $\tilde{\mathbf{X}}_i \in \mathbb{R}^{N_i \times d}$ denote the *uncentered* feature matrix at node i , and let $\tilde{\mathbf{X}}_{i, \cdot, j} \in \mathbb{R}^{N_i}$ be its j -th column. Define a boolean mask $\mathbf{m}_L = (\tilde{\mathbf{X}}_{i, \cdot, j} \leq t)$. The raw subsets are generated via logical indexing:

$$\begin{aligned} \tilde{\mathbf{X}}_L &= \tilde{\mathbf{X}}_i[\mathbf{m}_L] \in \mathbb{R}^{N_L \times d}, & \tilde{\mathbf{y}}_L &= \tilde{\mathbf{y}}_i[\mathbf{m}_L] \in \mathbb{R}^{N_L} \\ \tilde{\mathbf{X}}_R &= \tilde{\mathbf{X}}_i[\neg \mathbf{m}_L] \in \mathbb{R}^{N_R \times d}, & \tilde{\mathbf{y}}_R &= \tilde{\mathbf{y}}_i[\neg \mathbf{m}_L] \in \mathbb{R}^{N_R} \end{aligned}$$

where $N_L + N_R = N_i$. The corresponding child nodes $\mathcal{N}_L$ and $\mathcal{N}_R$ are then constructed from these subsets (including recomputing their own means and centered matrices).

The algorithm seeks the optimal split parameters (j^*, t^*) that minimize the combined impurity of the two child nodes:

$$(j^*, t^*) = \arg \min_{j \in \{1, \dots, d\}, t \in \mathbb{R}} [L^*(\mathcal{N}_L) + L^*(\mathcal{N}_R)] \quad (7)$$

Stopping Criteria. Recursive splitting terminates, finalizing a node as a leaf, if any of the following hold:

1. The hyperparameter `max_depth` is reached.
2. The optimal split does not decrease the loss, i.e., the condition

$$L^*(\mathcal{N}_L) + L^*(\mathcal{N}_R) < L^*(\mathcal{N}_i)$$

evaluates to **False**. In this case the split is rejected and the node is marked a leaf.

The Computational Trick for Efficient Splitting

Naively recomputing the matrix inversion $(\mathbf{X}_L^T \mathbf{X}_L + \lambda I)^{-1}$ for every candidate threshold t requires $\mathcal{O}(N_i^2 d^3)$ operations in total. A sequential-update trick reduces this to $\mathcal{O}(N_i \log N_i + N_i d^2)$ per feature.

To evaluate all thresholds for a fixed feature j :

1. **Sort.** Order all N_i examples by their value of feature j in ascending order.
2. **Initialize Running Statistics.** Maintain a state tuple $(n, \bar{\mathbf{x}}, \bar{y}, S_{xx}, S_{xy}, S_{yy})$ for both D_L (initially empty, so $n = 0$ and all statistics are zero) and D_R (initially containing all N_i points with statistics computed from D_i).
3. **Sequential Shift.** As the threshold sweeps across sorted values, exactly one data point $(\mathbf{x}^{(k)}, y^{(k)})$ moves from D_R to D_L at each step.
4. **Update Rules.** When point $(\mathbf{x}^{(k)}, y^{(k)})$ is added to D_L (which currently has n_{old} points), the statistics update as follows. Let $n_{\text{new}} = n_{\text{old}} + 1$, $\delta_x = \mathbf{x}^{(k)} - \bar{\mathbf{x}}_{\text{old}}$, and $\delta_y = y^{(k)} - \bar{y}_{\text{old}}$.

$$n_{\text{new}} = n_{\text{old}} + 1 \quad (8)$$

$$\bar{\mathbf{x}}_{\text{new}} = \bar{\mathbf{x}}_{\text{old}} + \frac{\delta_x}{n_{\text{new}}} \quad (9)$$

$$\bar{y}_{\text{new}} = \bar{y}_{\text{old}} + \frac{\delta_y}{n_{\text{new}}} \quad (10)$$

$$S_{xx,\text{new}} = S_{xx,\text{old}} + \frac{n_{\text{old}}}{n_{\text{new}}} \delta_x \delta_x^T \quad (11)$$

$$S_{xy,\text{new}} = S_{xy,\text{old}} + \frac{n_{\text{old}}}{n_{\text{new}}} \delta_y \delta_x \quad (12)$$

$$S_{yy,\text{new}} = S_{yy,\text{old}} + \frac{n_{\text{old}}}{n_{\text{new}}} \delta_y^2 \quad (13)$$

Symmetric *subtractive* counterparts of Equations (8)–(13) apply when removing the same point from D_R . The derivation of Equation (11) is given in Appendix B.

Because S_{xx} undergoes a rank-1 outer-product update, its inverse can be maintained in $\mathcal{O}(d^2)$ time per step via the **Sherman–Morrison formula**:

$$(A + \mathbf{u}\mathbf{u}^T)^{-1} = A^{-1} - \frac{A^{-1}\mathbf{u}\mathbf{u}^T A^{-1}}{1 + \mathbf{u}^T A^{-1} \mathbf{u}}$$

applied with $A = S_{xx,\text{old}} + \lambda I$ and $\mathbf{u} = \sqrt{\frac{n_{\text{old}}}{n_{\text{new}}}} \delta_x$.

Inference

To compute predictions for a test matrix $\mathbf{X}_{\text{new}} \in \mathbb{R}^{M \times d}$ after training:

1. **Routing.** For each test row $\mathbf{x}_{\text{new}}^{(m)} \in \mathbb{R}^d$, traverse the tree from the root. At each internal node with split (j, t) : go left if $x_{\text{new},j}^{(m)} \leq t$, otherwise go right.

2. **Leaf Retrieval.** Upon reaching a leaf node $\mathcal{N}_{\text{leaf}}$, retrieve its frozen parameters $(\mathbf{w}_{\text{leaf}}, \bar{\mathbf{x}}_{\text{leaf}}, \bar{y}_{\text{leaf}})$.
3. **Vectorized Prediction.** Let K be the number of test points routed to $\mathcal{N}_{\text{leaf}}$, and let $\mathbf{X}_{\text{leaf}} \in \mathbb{R}^{K \times d}$ be the corresponding submatrix of $\mathbf{X}_{\text{new}}$. The localized predictions are:

$$\hat{\mathbf{y}}_{\text{leaf}} = (\mathbf{X}_{\text{leaf}} - \mathbf{1}\bar{\mathbf{x}}_{\text{leaf}}^T) \mathbf{w}_{\text{leaf}} + \bar{y}_{\text{leaf}} \mathbf{1} \in \mathbb{R}^K \quad (14)$$

where $\mathbf{1} \in \mathbb{R}^K$ is an all-ones column vector and $\bar{\mathbf{x}}_{\text{leaf}}^T$ is broadcast across all K rows.

Task 1: The Buggy Ride

You are provided with a skeleton implementation of the `LinearRegressionTree` class using NumPy. The code contains logical and mathematical errors within the update rules and tree traversal.

Your task is to use the function docstrings and the mathematical definitions above to debug the implementation. You must modify the source code such that running the testing script `debug_check.py` passes all unit tests. At a high level, bugs are of two types, syntactic and silent bugs. We will have weightages for both of them. Also there would be one bug which `debug_check` fails to catch at all (i.e. 6/6 passes but still incorrect), but might affect your answer in task2, so beware of it.

Note: A good idea would be to first jot down only the very relevant steps/equations that need to go into the code with the correct shapes and all. Then go over every function in the class and figure what is not correct or perhaps missing. If you are unsure of any existing code, please lookup the documentation.

Task 2: Split Segments

Once the framework is correct, you will evaluate it on the non-linear **Segment** dataset, which features complex piecewise linear trends that benefit from local estimators.

Your task is to tune the hyperparameters to minimize test error:

1. Run `main.py` to train the model and generate prediction boundary plots.
2. Use the visual and diagnostic outputs to manually adjust `max_depth` and λ .
3. Hardcode your final configuration in the `give_optimal_hyperparameter` function in `tree.py`.

Grading for this task is quantitatively based on the Mean Squared Error (MSE), R^2 values on unseen public and private test splits.

Q3. What's in a Model? [35 marks]

Willie Wonka has been running a new magic chocolate machine for the last few weeks. This machine can make two types of magical chocolate: "Gopstopper" and "Wonkagum". He wants to analyse this new machine to better understand how it works.

Interestingly, the machine does not let Wonka explicitly decide which type of chocolate to produce. Instead, it automatically spits out one of the two types of chocolate based on the internal conditions present during the manufacturing process. Fortunately, Wonka has identified the key factors that influence this split. He spent the first few weeks collecting data on when the machine produced which type of chocolate. This has been given to you in dataset 1 (q3_1.csv).

While making chocolate, the machine also consumes energy. Wonka wants to investigate the factors that lead to energy consumption in the machine. He spent his remaining time collecting some additional information which he thinks might help in determining the energy use by the machine. This has been given to you in dataset 2 (q3_2.csv). The columns in the second dataset are a superset of the columns in the first.

Your task in this problem is the following:

Task 1: Classification Model

Begin first by training a classification model to predict the "ChocolateType" column using the data in q3_1.csv using the appropriate library functions in sklearn. While a naive fit will still give you some predictive power, try to think about the techniques you have learned about throughout the course till now, and which among them may be relevant in improving the accuracy of this model.

Even a naive fit here gets roughly an accuracy of 80%. If you can get $> 97\%$ accuracy then you can consider this task complete.

Task 2: Building the Regressor

This is one of the more open-ended tasks in this paper, so you are recommended to not spend too much time here until you are done with the previous parts.

Now have a look at q3_2.csv. Your job is to train a model to predict the energy consumption. While you are restricted to using only the library functions imported in the `model.py` code, this does not mean that your final prediction function must be purely linear. You can engineer features of any kind you want in both training and prediction. There can be significant gains in the final accuracy if you go beyond a naive model and incorporate more information into your regression model.

For this part, it may be helpful to do some exploratory data analysis using pandas dataframes. To help with that, we have provided a sample solution to the EDA section of Lab-0. You can use it to see the syntax for things you may want to do with pandas in a Jupyter notebook. In addition to that, an `analyze_dataframes` function is provided in the solve code itself. You can call it on a pandas dataframe to interactively save plots regarding the dataframe.

Submission Instructions

Submit the model.py file as a subdirectory in q3.

Helpful Notes

The evaluation for this question will be based solely on the accuracies of the models on a hidden validation dataset (which is drawn from the same distribution as the test and train dataset).

For this question (And **THIS QUESTION ONLY**). You are allowed and encouraged to use the sklearn library functions imported at the beginning.

Please **DO NOT** add any additional imports. **your solution will not be evaluated if there are any additional imports found.**

If you feel stuck try answering the following questions:

1. Which of the columns do I expect to have predictive power?
2. How can I regularize my model?
3. What kind of features should I try adding?
4. How can I use the fact that both the datasets have many shared columns?
5. Is there any transform that I can do to make this feature more suitable for a linear model?

Please do not blindly add features it is often better to have a closer look at the data and to reason properly about the type of features that you expect to work well.

Rough guideline on what accuracy to aim for

All of these accuracies are quoted for the provided test train split. You can of-course get much better accuracy (even 100%!) if your model cheats and looks at the test data when it isn't supposed to.

Only the first digit is really significant here; your mileage may vary by a few percentage points. Please do not worry about that.

1. Classification (naive): 80 % Accuracy (5% weightage)
2. Classification (with some Optimizations): 97 % or above Accuracy (30 % weightage)
3. Regression (naive): R^2 score of: 50% (5 % weightage)
4. Regression (with some optimizations): R^2 score of: 90% (40 % weightage)
5. Regression (with good optimizations and feature engineering): R^2 score of: 98% (20 % weightage)

The weightage is cumulative. For eg. If you get a 98% accuracy in your classification task then you will definitely get 35% of the grade. For intermediate values for e.g. an accuracy of 85% the grading will be determined later.

Please note that the final few optimizations may require noticing a few small things regarding the data. I would advise you to try to go beyond bullet 4 only after you are fairly confident that the rest of the paper is done.

A Simplified Optimal Loss

Claim. At the minimizer $\mathbf{w}_i = (S_{xx} + \lambda I)^{-1} S_{xy}$, the minimum loss equals $L^*(\mathcal{N}_i) = S_{yy} - \mathbf{w}_i^T S_{xy}$.

Proof. Expand the vectorized loss:

$$L(\mathbf{w}) = \mathbf{y}^T \mathbf{y} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T (\mathbf{X}^T \mathbf{X} + \lambda I) \mathbf{w}.$$

Substituting $\mathbf{w}_i = (\mathbf{X}^T \mathbf{X} + \lambda I)^{-1} \mathbf{X}^T \mathbf{y}$ into the last term:

$$\mathbf{w}_i^T (\mathbf{X}^T \mathbf{X} + \lambda I) \underbrace{(\mathbf{X}^T \mathbf{X} + \lambda I)^{-1} \mathbf{X}^T \mathbf{y}}_{=\mathbf{w}_i} \cdot (\mathbf{X}^T \mathbf{X} + \lambda I) (\mathbf{X}^T \mathbf{X} + \lambda I)^{-1} = I,$$

so the last term simplifies to $\mathbf{w}_i^T \mathbf{X}^T \mathbf{y}$. Hence:

$$L^* = \mathbf{y}^T \mathbf{y} - 2\mathbf{w}_i^T \mathbf{X}^T \mathbf{y} + \mathbf{w}_i^T \mathbf{X}^T \mathbf{y} = \mathbf{y}^T \mathbf{y} - \mathbf{w}_i^T \mathbf{X}^T \mathbf{y} = S_{yy} - \mathbf{w}_i^T S_{xy}.$$

B Rank-1 Covariance Update

Claim. When adding the n -th point $\mathbf{x}^{(n)}$ to a running set of $n - 1$ points, the centered second-moment matrix updates as:

$$S_{xx,n} = S_{xx,n-1} + \frac{n-1}{n} \boldsymbol{\delta} \boldsymbol{\delta}^T, \quad \boldsymbol{\delta} = \mathbf{x}^{(n)} - \bar{\mathbf{x}}_{n-1}.$$

Proof. By definition,

$$S_{xx,n} = \sum_{i=1}^n (\mathbf{x}^{(i)} - \bar{\mathbf{x}}_n)(\mathbf{x}^{(i)} - \bar{\mathbf{x}}_n)^T.$$

Separating the n -th term:

$$S_{xx,n} = \underbrace{\sum_{i=1}^{n-1} (\mathbf{x}^{(i)} - \bar{\mathbf{x}}_n)(\mathbf{x}^{(i)} - \bar{\mathbf{x}}_n)^T}_{(A)} + \underbrace{(\mathbf{x}^{(n)} - \bar{\mathbf{x}}_n)(\mathbf{x}^{(n)} - \bar{\mathbf{x}}_n)^T}_{(B)}.$$

From the mean update rule (9) applied at step n :

$$\bar{\mathbf{x}}_n = \bar{\mathbf{x}}_{n-1} + \frac{\boldsymbol{\delta}}{n}.$$

For any $i < n$, the centered term expands as:

$$\mathbf{x}^{(i)} - \bar{\mathbf{x}}_n = (\mathbf{x}^{(i)} - \bar{\mathbf{x}}_{n-1}) - \frac{\boldsymbol{\delta}}{n}.$$

Expanding term (A) and noting that $\sum_{i=1}^{n-1} (\mathbf{x}^{(i)} - \bar{\mathbf{x}}_{n-1}) = \mathbf{0}$ (cross terms vanish):

$$(A) = S_{xx,n-1} + \frac{n-1}{n^2} \boldsymbol{\delta} \boldsymbol{\delta}^T.$$

For term (B): $\mathbf{x}^{(n)} - \bar{\mathbf{x}}_n = \boldsymbol{\delta} - \frac{\boldsymbol{\delta}}{n} = \frac{n-1}{n}\boldsymbol{\delta}$, so

$$(B) = \frac{(n-1)^2}{n^2} \boldsymbol{\delta} \boldsymbol{\delta}^T.$$

Summing (A) and (B):

$$S_{xx,n} = S_{xx,n-1} + \left(\frac{n-1}{n^2} + \frac{(n-1)^2}{n^2} \right) \boldsymbol{\delta} \boldsymbol{\delta}^T = S_{xx,n-1} + \frac{n-1}{n} \boldsymbol{\delta} \boldsymbol{\delta}^T.$$

By analogous arguments, the updates for S_{xy} and S_{yy} (Equations (12)–(13)) follow identically with $\boldsymbol{\delta}$ and δ_y in place of the two $\boldsymbol{\delta}$ factors.